

Offroad Semantic Segmentation using SegFormer-B2 with Correct Label Remapping

Haris Bin Nasir and Abdullah Agha

Team Epoch & Chill

February 15, 2026

Abstract

This report describes a semantic segmentation pipeline for offroad scene understanding using SegFormer-B2. The dataset provides segmentation masks with sparse raw label IDs (e.g., 100, 200, ..., 10000), requiring a strict remapping to contiguous training indices for learning and an inverse mapping at inference time for generating valid submission masks. The approach includes ImageNet normalization, strong but efficient augmentations, class-weighted cross-entropy combined with Dice loss, mixed precision training (AMP), and test-time augmentation (multi-scale + horizontal flip). Performance is evaluated using mean Intersection over Union (mIoU) on the validation split and test_public_80.

1 Introduction

Semantic segmentation assigns a semantic label to each pixel and is a core capability for autonomous navigation and robotics in unstructured environments. Offroad imagery is challenging due to diverse textures, lighting, and terrain appearance. We use SegFormer-B2 because transformer-based encoders frequently generalize better to unseen environments than classic CNN-only decoders while maintaining practical runtime.

2 Dataset and Label Remapping

The dataset contains RGB images and corresponding segmentation masks. Mask pixel values represent class IDs using a sparse set of integers rather than contiguous indices. Training requires labels to be in the range $\{0, \dots, C - 1\}$, so we remap raw IDs to contiguous training indices. Any unexpected pixel values are ignored using an ignore index.

2.1 Class ID Mapping

Table 1: Raw mask IDs and contiguous training indices.

Raw Mask ID	Train Index
100	0
200	1
300	2
500	3
550	4
600	5
700	6
800	7
7100	8
10000	9

2.2 Dataset Examples



Figure 1: RGB input image (left) and corresponding ground-truth segmentation mask (right).

3 Method

3.1 Model

We fine-tune `nvidia/segformer-b2-finetuned-ade-512-512` by replacing its classification head to output 10 classes. Since ADE20K uses 150 classes, the segmentation head weights are re-initialized due to shape mismatch and learned during training.

3.2 Preprocessing and Augmentation

All images are resized to 512×512 and normalized with ImageNet mean and standard deviation. Augmentations were selected for generalization while remaining stable for segmentation:

- Random horizontal flip
- Shift-scale-rotate
- Random brightness/contrast and hue/saturation adjustments
- Gaussian noise

3.3 Loss Function

We optimize the sum of class-weighted cross entropy and Dice loss:

$$\mathcal{L} = \mathcal{L}_{CE} + \mathcal{L}_{Dice}$$

Class weights are computed from a subset of training masks after remapping, improving robustness under class imbalance. Dice loss encourages better overlap and helps refine boundaries.

3.4 Evaluation Metric

We report mean Intersection over Union (mIoU), computed over classes present in the evaluation batch, ignoring pixels assigned to the ignore index.

4 Implementation

Training uses PyTorch with mixed precision (AMP) and AdamW optimizer. A polynomial learning rate schedule is applied:

$$lr = lr_0 \left(1 - \frac{epoch}{E}\right)^{0.9}.$$

Key hyperparameters:

- Image size: 512
- Batch size: 8
- Learning rate: 6×10^{-5}
- Weight decay: 1×10^{-4}
- Epochs: 45 (best checkpoint selected by validation mIoU)

4.1 Key Implementation Snippets

Listing 1: Raw-to-train remapping used in training dataloader.

Raw-to-train remapping (training)

```
1 RAW_CLASS_IDS = [100,200,300,500,550,600,700,800,7100,10000]
2 RAW2TRAIN = {rid:i for i,rid in enumerate(RAW_CLASS_IDS)}
3 IGNORE_INDEX = 255
4
5 def remap_mask_raw_to_train(mask_raw):
6     out = np.full(mask_raw.shape, IGNORE_INDEX, dtype=np.uint8)
7     for rid, tid in RAW2TRAIN.items():
8         out[mask_raw == rid] = tid
9     return out
```

Listing 2: Inverse mapping used for generating uint16 raw-ID masks for submission.

Train-to-raw remapping (submission masks)

```
1 TRAIN2RAW = {i:rid for i,rid in enumerate(RAW_CLASS_IDS)}
2
3 def remap_mask_train_to_raw(mask_train):
4     out = np.zeros(mask_train.shape, dtype=np.uint16)
5     for tid, rid in TRAIN2RAW.items():
6         out[mask_train == tid] = rid
7     return out
```

Listing 3: TTA with resizing probability maps to a common spatial size before averaging.
Test-time augmentation (multi-scale + flip)

```

1 probs_all = []
2 base_hw = None
3 for s in scales:
4     probs = softmax(model(pixel_values=x).logits) # shape: [1,C,h,w]
5     if base_hw is None:
6         base_hw = probs.shape[-2:]
7     probs = interpolate(probs, size=base_hw) # align shapes
8     probs_all.append(probs)
9 final_probs = mean(stack(probs_all))
10 pred = argmax(final_probs)

```

5 Results

5.1 Training Curves

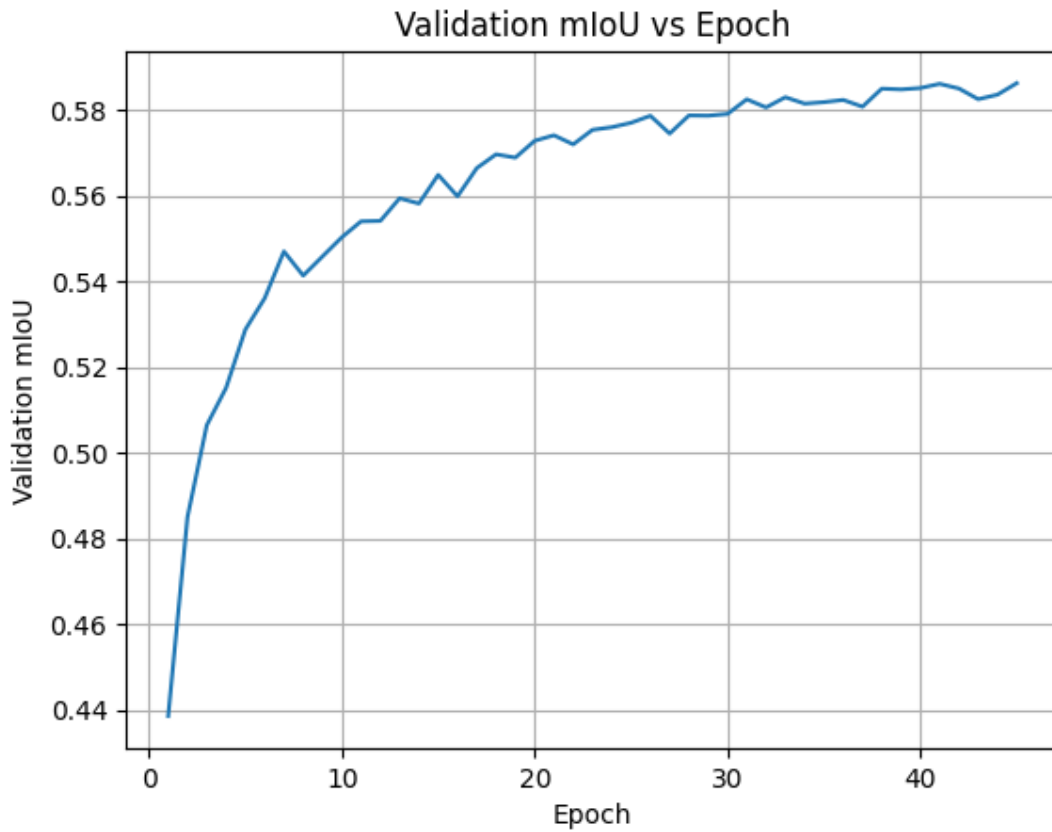


Figure 2: Validation mIoU across training epochs.

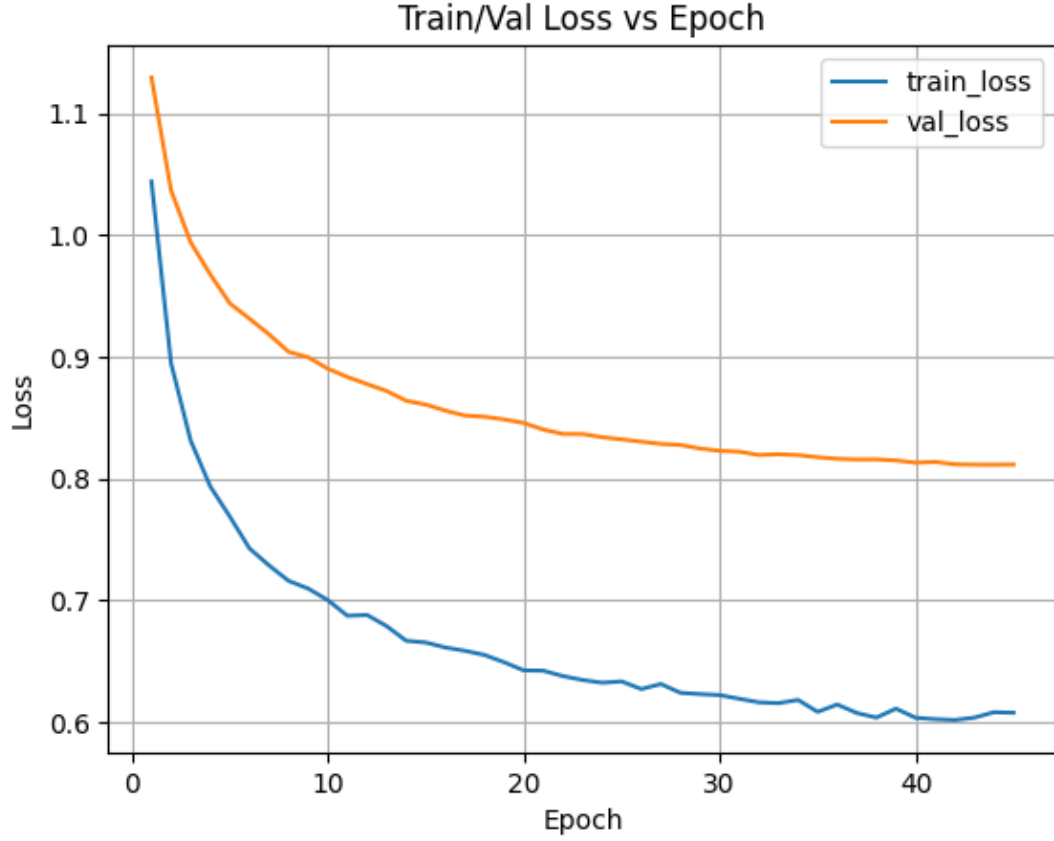


Figure 3: Training and validation loss across epochs.

5.2 Qualitative Predictions

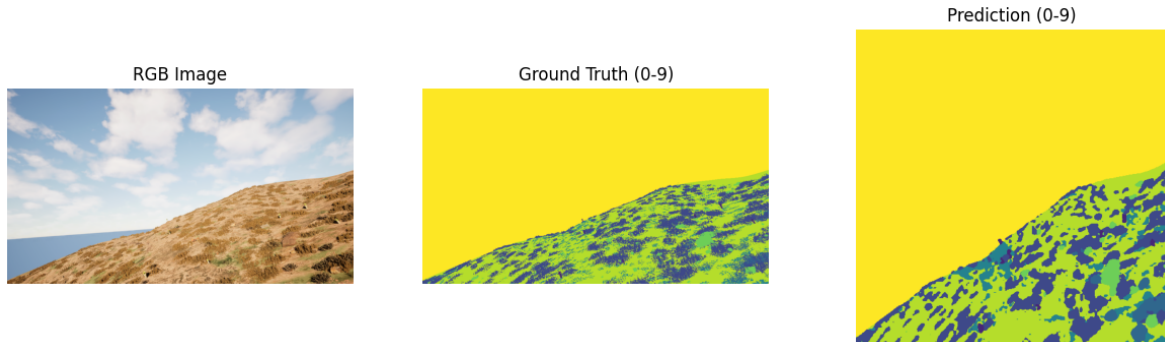


Figure 4: Qualitative example: input image, ground truth mask, and predicted mask (raw IDs).

5.3 Quantitative Metrics

Table 2: Final evaluation metrics.

Metric	Score
Validation mIoU (best epoch)	[0.59]
test_public_80 mIoU	[0.39]

6 Submission Artifacts

The submission package contains:

- `train.py`: training script with label remapping, losses, AMP, and checkpointing.
- `test.py`: inference script that loads `model.pkl` and outputs raw-ID masks and a ZIP file.
- `model.pkl`: serialized model bundle containing the model name, weights, class IDs, and normalization stats.
- `requirements.txt`: required dependencies for reproducibility.

7 Conclusion

This work delivers an offroad semantic segmentation solution based on SegFormer-B2. Correct label remapping was essential to obtain valid learning signals and submission-ready masks. The combination of class-weighted cross-entropy, Dice loss, augmentation, AMP training, and test-time augmentation improved stability and generalization. Future improvements may include higher-resolution fine-tuning, stronger backbones (SegFormer-B3/B4), and lightweight post-processing for boundary refinement.